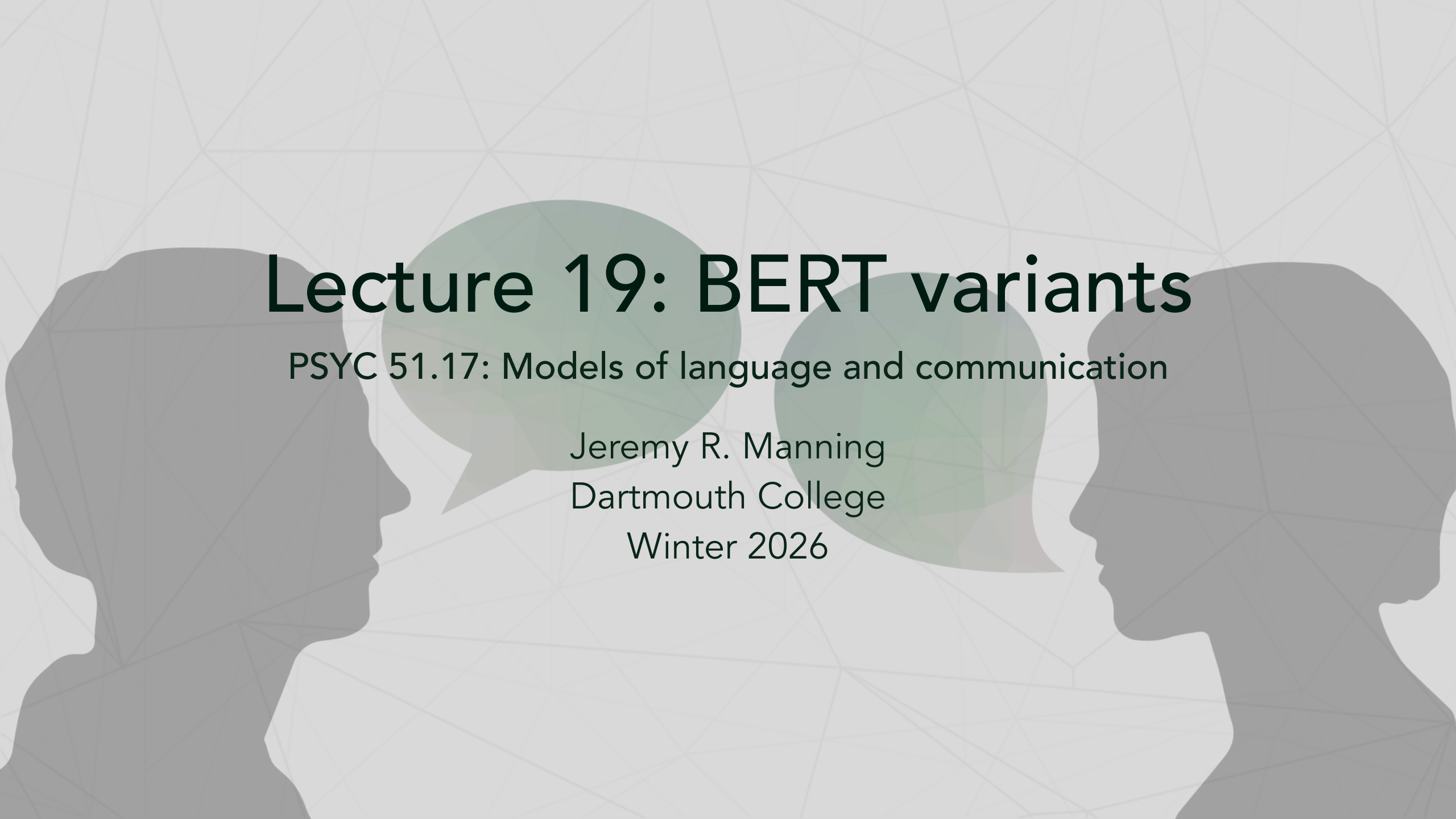


Lecture 19: BERT variants

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Identify key limitations of the original BERT training procedure
2. Explain how RoBERTa, ALBERT, DistilBERT, ELECTRA, and DeBERTa each address different limitations
3. Compare parameter efficiency, training efficiency, and inference speed across variants
4. Select the appropriate variant for a given task and resource constraint
5. Argue whether encoder models remain relevant in the era of GPT-4

What could be improved?

BERT's key limitations (from Lecture 18)

Training procedure: NSP may hurt performance; static masking reuses the same masks every epoch; only 15% of tokens provide training signal (recall the 80/10/10 MLM procedure).

Scale: Trained on only 3.3B words with 100K steps — modern datasets are 100× larger (and often train for much longer).

Efficiency: 110M parameters are all active for every input. Large memory footprint for deployment.

Companion notebook

Companion Notebook — try different BERT variants hands-on

RoBERTa: robustly optimized BERT

Key idea: better training = better performance

RoBERTa (Liu et al., 2019) keeps BERT's architecture but fixes the training recipe:

1. **Remove NSP** — Next Sentence Prediction hurt performance. Use only MLM with full sentences.
2. **Dynamic masking** — Generate a new masking pattern every time a sequence is seen, instead of reusing the same mask.
3. **Larger batches, more data** — Batch size 8K sequences (vs BERT's 256), 160GB text (vs 16GB), 500K steps (vs 100K).
4. **Longer sequences** — Train on longer contiguous text for better long-range understanding.

Takeaway

Training procedure matters as much as architecture. RoBERTa shows that BERT was significantly **undertrained**.

Dynamic masking

RoBERTa's key innovation

BERT used **static masking** — the same mask positions every epoch, risking memorization. RoBERTa generates **new masks on-the-fly** during training:

- Epoch 1: "My [MASK] is cute"
- Epoch 2: "My dog [MASK] cute"
- Epoch 3: "My dog is [MASK]"

More diverse training signal → better generalization. Combined with removing NSP and training longer on more data, this alone accounts for most of RoBERTa's gains.

RoBERTa results

Consistent improvements over BERT

Task (metric)	BERT-Large	RoBERTa	Improvement
SQuAD 2.0 (F1)	83.1	**89.4**	+6.3
MNLI (accuracy)	86.7	**90.2**	+3.5
SST-2 (accuracy)	94.9	**96.4**	+1.5
RACE (accuracy)	72.0	**83.2**	+11.2

Understanding the benchmarks

SQuAD 2.0: reading comprehension + unanswerable questions (human F1: 89.5 — RoBERTa essentially matches humans). **MNLI**: natural language inference across 10 genres (human: 92.0%, random: 33.3%). **SST-2**: binary sentiment classification (human: ~97%, random: 50%). **RACE**: multiple-choice reading comprehension from English exams (human: 92.2%, random: 25%). The RACE gain (+11.2 pts) is especially striking because it requires multi-sentence reasoning — exactly the capability that better training unlocks.

Key findings

- Dynamic masking consistently outperforms static masking
- More data + longer training = substantial gains
- Removing NSP improves downstream task performance
- Some tasks see enormous gains (RACE: +11.2 points!)

ALBERT: a lite BERT

Key idea: parameter sharing for efficiency

ALBERT (Lan et al., 2019) dramatically reduces BERT's parameter count through three innovations:

1. Factorized embedding parameters:

- BERT: vocabulary (30K) × hidden size (768) = 23M parameters
- ALBERT: vocabulary (30K) × embedding size (128) + embedding (128) × hidden (768) = 3.9M parameters
- **83% fewer embedding parameters**

2. Cross-layer parameter sharing:

- All 12 transformer layers share the *same* weights
- Like running the same layer 12 times (iterative refinement)
- **89% fewer transformer parameters**

3. Sentence Order Prediction (SOP):

- Replaces NSP with a harder task: are sentences A, B in the correct order?
- Forces the model to learn discourse coherence, not just topic matching

Factorized embedding parameters

Decomposing a large matrix into two small ones

BERT embeds each token directly into the hidden dimension $C = 768$. ALBERT introduces a bottleneck $E = 128 \ll C$:

$$\underbrace{V \times C}_{\text{BERT: 30K}\times\text{768}} \longrightarrow \underbrace{V \times E}_{\text{30K}\times\text{128}} \times \underbrace{E \times C}_{\text{128}\times\text{768}}$$

This factorization applies to the **token embeddings**, which dominate the parameter count. Positional ($512 \times E$) and segment ($2 \times E$) embeddings also use E, but they're tiny. All three are summed in E-space, then projected to C with a single linear layer.

Why this works

The vocabulary matrix is low-rank — tokens cluster into a much smaller subspace than $C = 768$. Embeddings only need to encode **token identity**; the projection layer handles the mapping to hidden space. Result: **83% fewer embedding parameters** (23M → 3.9M).

ALBERT parameter efficiency

Per-model parameter counts

Model	Layers	Hidden size	Parameters
BERT-base	12	768	110M
ALBERT-base	12	768	**12M**
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Factorized embedding in Python

Try it yourself!

```
import torch.nn as nn

# BERT: direct embedding (30K vocab × 768 hidden = 23M params)
bert_embed = nn.Embedding(30000, 768)

# ALBERT: two-step embedding (30K × 128 + 128 × 768 = 3.9M params)
albert_embed = nn.Embedding(30000, 128)
albert_project = nn.Linear(128, 768)
# 83% fewer embedding parameters!
```

Cross-layer parameter sharing: BERT vs ALBERT

BERT layer structure

```
class BERT:
    def __init__(self):
        # Each layer has unique parameters
        self.layers = [TransformerLayer() for _ in range(12)]
        # 12 × 7M params = 85M params in transformer layers

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)   # Different weights each time
        return x
```

Cross-layer parameter sharing: BERT vs ALBERT

ALBERT layer structure

```
class ALBERT:
    def __init__(self):
        # Single shared layer
        self.shared_layer = TransformerLayer()
        # 1 × 7M params = 7M params (89% reduction!)

    def forward(self, x):
        for _ in range(12):
            x = self.shared_layer(x)  # Same weights reused
        return x
```

Trade-off

ALBERT has 89% fewer parameters but the **same compute cost** — it still performs 12 forward passes through the transformer layer. The savings are in memory, not speed.

DistilBERT: knowledge distillation

Key idea: train a small model to mimic a large model

Knowledge distillation (Sanh et al., 2019) compresses BERT into a smaller, faster model:

- **Teacher**: Full BERT-base (12 layers, frozen)
- **Student**: DistilBERT (6 layers, trainable)
- **Training signal**: Student learns to match the teacher's *soft probability distributions*, not just the hard labels

The teacher's "wrong" predictions contain useful information — e.g., predicting "cat" is more likely than "car" for a masked animal slot tells the student about semantic similarity.

Results

Metric	BERT-base	DistilBERT	Change
Parameters	110M	66M	40% smaller
Inference speed	1×	1.6×	60% faster
GLUE score	79.6	77.0	97% retained

What is GLUE?

GLUE (General Language Understanding Evaluation) is a benchmark suite of 9 tasks — including MNLI, SST-2, and others — that tests grammar, sentiment, similarity, and inference. The score is an average across all tasks; human baseline is ~87.

DistilBERT training in Python

Knowledge distillation training loop

```
teacher = BertModel.from_pretrained("bert-base")  # 12 layers, frozen
student = DistilBertModel(num_layers=6)            # 6 layers, trainable

for batch in training_data:
    # Teacher provides "soft targets" (probability distributions)
    with torch.no_grad():
        teacher_logits = teacher(batch)  # e.g., [0.7, 0.2, 0.1, ...]

    # Student tries to match teacher's distribution
    student_logits = student(batch)

    # Distillation loss: KL divergence between soft distributions
    # Temperature T=2 softens the distribution (more informative)
    loss_distill = KL_divergence(
        softmax(student_logits / T),
        softmax(teacher_logits / T)
    )
    loss_mlm = masked_lm_loss(student_logits, labels)

    loss = 0.5 * loss_distill + 0.5 * loss_mlm
```

ELECTRA: efficient learning from *all* tokens

Key idea: learn from 100% of tokens, not just 15%

ELECTRA (Clark et al., 2020) uses a **generator-discriminator** setup:

1. A small **generator** (like a mini-BERT) fills in masked tokens with plausible replacements
2. A **discriminator** classifies *every* token as original or replaced
3. The discriminator provides a training signal for **all tokens** — not just the 15% that were masked

Why this is harder (and better) than MLM

BERT's MLM replaces tokens with `[MASK]` — an obvious tell that never appears in real text. The discriminator's task is harder: the generator produces *plausible* substitutions ("ate" for "cooked"), so the discriminator must understand the full context deeply enough to detect subtle semantic mismatches. This is closer to how humans process language — we don't spot blank slots, we notice when something *doesn't quite fit*.

ELECTRA in Python

Complete example — paste into Colab and run!

```python
import torch
from transformers import AutoTokenizer, AutoModelForMaskedLM, ElectraForPreTraining

tokenizer = AutoTokenizer.from_pretrained("google/electra-small-generator")
generator = AutoModelForMaskedLM.from_pretrained("google/electra-small-generator")
discriminator = ElectraForPreTraining.from_pretrained("google/electra-small-discriminator")

# Step 1: Mask a token and let the generator fill it in
original = "The chef cooked a delicious meal"
masked = "The chef [MASK] a delicious meal"
inputs = tokenizer(masked, return_tensors="pt")

with torch.no_grad():
    gen_logits = generator(**inputs).logits

mask_idx = (inputs.input_ids == tokenizer.mask_token_id).nonzero(as_tuple=True)[1]
predicted_id = gen_logits[0, mask_idx].argmax(dim=-1)
replacement = tokenizer.decode(predicted_id)
print(f"Generator filled [MASK] → '{replacement}'")  # e.g., "prepared"

# Step 2: Discriminator classifies EVERY token as original or replaced
fake_ids = inputs.input_ids.clone()
fake_ids[0, mask_idx] = predicted_id
with torch.no_grad():
    disc_logits = discriminator(fake_ids).logits

```

ELECTRA efficiency

Learning from every token

ELECTRA learns from **100% of tokens** (every position gets a real/replaced label), compared to BERT's 15%. This 6.7× increase in training signal means ELECTRA reaches BERT-level performance with **4× less compute**.

When to use ELECTRA

ELECTRA is ideal when you have limited compute budget:

- ELECTRA-Small outperforms BERT-Small
- ELECTRA-Base is competitive with BERT-Large
- With the same compute budget, ELECTRA consistently wins

ModernBERT: 6 years of decoder tricks, applied to encoders

Key idea: modernize the encoder with techniques from the GPT era

ModernBERT (Warner et al., 2024) asks: what if we rebuilt BERT from scratch using everything we've learned from training decoders?

Component	BERT (2018)	ModernBERT (2024)
Position encoding	Learned absolute (512 max)	**RoPE** (8,192 tokens)
Attention	Full quadratic	**Flash Attention** + alternating global/local
Padding	Processes pad tokens	**Unpadding** (only real tokens)
Training data	3.3B words	**2 trillion tokens** (600×)
Code understanding	None	Trained on code corpora

Results

SOTA on GLUE, retrieval (MTEB), and code understanding. Available as `answerdotai/ModernBERT-base` and `answerdotai/ModernBERT-large`. Proves that the encoder architecture still has room to grow.

ModernBERT key innovations

RoPE (Rotary Position Embeddings)

Instead of learning a fixed position embedding for each slot (BERT's approach, capped at 512 tokens), RoPE encodes position by **rotating** the query and key vectors in attention. Relative distances are captured by the angle between rotated vectors. This generalizes to sequences longer than training length — ModernBERT handles **8,192 tokens** vs BERT's 512.

Flash Attention

Standard attention materializes the full $T \times T$ attention matrix in GPU memory ($O(T^2)$ space). Flash Attention computes attention **tile-by-tile** in fast on-chip SRAM, never storing the full matrix. Same exact result, but ~2–4× faster and uses $O(T)$ memory. ModernBERT alternates between global attention (every token sees every token) and local attention (sliding window) across layers.

Unpadding

Batched inputs require padding shorter sequences to the same length. BERT wastes compute processing these pad tokens through every layer. Unpadding **strips** pad tokens before the transformer and **reinserts** them after, so the model only processes real tokens. In batches with variable-length inputs, this can save 20–30% of total compute.

Variant comparison summary

Choosing the right BERT variant

Model	Key innovation	Best for
BERT	MLM + NSP	Baseline, well-understood
RoBERTa	Better training recipe	Maximum quality (classic)
ALBERT	Parameter sharing	Memory-constrained deployment
DistilBERT	Knowledge distillation	Speed-critical production
ELECTRA	Replaced token detection	Limited training budget
ModernBERT	Modern training + RoPE + Flash Attention	Maximum quality (2024)

General guidelines

- **Best quality (2024):** ModernBERT-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT
- **Limited training budget:** ELECTRA
- **Good default:** RoBERTa-Base or ModernBERT-Base

Other notable BERT variants

The BERT family keeps growing

DeBERTa (Microsoft, 2020): Disentangled attention separates content and position representations. Enhanced mask decoder. State-of-the-art on SuperGLUE.

SpanBERT (Facebook, 2019): Masks random contiguous *spans* instead of individual tokens. Span boundary objective. Better for extractive tasks (QA, coreference).

ERNIE (Baidu, 2019): Entity-level and phrase-level masking. Knowledge-enhanced pre-training. Strong on Chinese NLP tasks.

BART (Facebook, 2019): Encoder-decoder architecture (not encoder-only). Denoising autoencoder with various corruption strategies. Excellent for generation tasks.

The case against encoders

Why did people start saying 'the encoder is dead'?

2020 — In-context learning: GPT-3 (Brown et al.) showed that a single decoder model can perform classification, NLI, and QA via prompting — tasks that previously required fine-tuning separate BERT models for each.

2023 — Decoders match fine-tuned encoders: GPT-4 (OpenAI) matched or exceeded fine-tuned BERT/RoBERTa on many NLU benchmarks without any task-specific training.

2024 — Decoders do retrieval too: GritLM (Muennighoff et al.) demonstrated a single decoder model that handles both generation *and* embedding at SOTA levels — the last domain where encoders had a clear advantage.

The argument in one sentence

Why fine-tune six BERT models for six tasks when one decoder does them all via prompting?

Gemma Encoder and the encoder renaissance

What is Gemma?

Gemma (Google, 2024) is a family of open-weight **decoder-only** language models (2B and 7B parameters) built from the same research behind Gemini. Outperforms similarly sized open models on 11/18 text benchmarks.

Google bets on encoders again (2025)

Gemma Encoder (2025) repurposes Gemma's decoder weights for **bidirectional** encoding — Google's first encoder-only model since BERT. Competitive with ModernBERT on sentence embedding tasks.

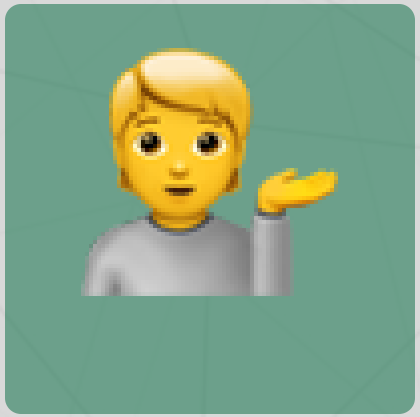
Why this matters

If Google — a company betting heavily on decoder-only models (Gemini) — still releases an encoder model, it signals that encoders serve a purpose decoders can't efficiently fill. The encoder isn't dead; it's being **modernized**.

Questions?

Email

Discord



Office hours

Up next...

Encoder models in the real world — applications, brain-model convergence, and what it all means for language and society